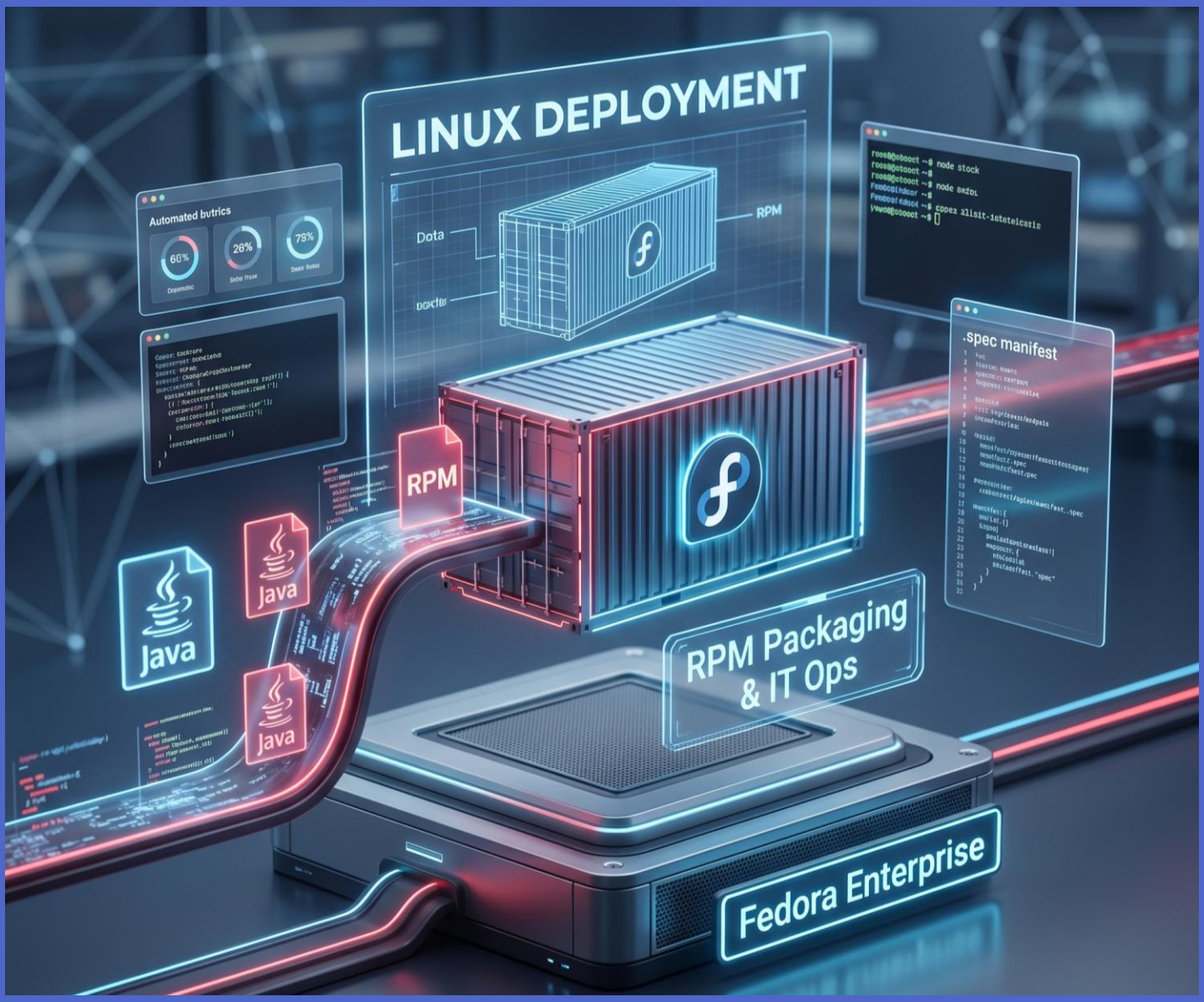
DESARROLLO DE INTERFACES

CASO PRACTICO I UD7



ALUMNO CESUR

25/26

Alejandro Muñoz de la Sierra

PROFESOR

Manuel Gómez Lora

I N T R O D U C C I O N

El presente caso práctico nace con un objetivo arquitectónico innegociable: abandonar definitivamente la ejecución manual, precaria y carente de trazabilidad de archivos binarios aislados (.jar), para dar el salto cualitativo hacia un modelo de despliegue profesional en entornos Linux de grado empresarial. No buscábamos simplemente "hacer que la aplicación funcionase" mediante parches temporales, sino interiorizar y replicar el flujo de trabajo estricto que sigue un equipo de Operaciones (IT Ops) cuando se enfrenta a la distribución masiva y segura de software.

Para lograr este hito, hemos aprovisionado desde cero un entorno de construcción (*build workspace*) aislado sobre la distribución Fedora Linux. En este entorno, configuramos meticulosamente cada herramienta de la *toolchain* de empaquetado y diseñamos un manifiesto declarativo (archivo .spec) capaz de transformar nuestro código base compilado en Java 21 en un paquete binario nativo (.rpm).

Nuestra intención desde el primer minuto ha sido abstraer la complejidad técnica del sistema. El objetivo final era garantizar que cualquier administrador de sistemas o usuario final pudiera instalar nuestra herramienta ejecutando un único comando transaccional, delegando en el gestor de paquetes la responsabilidad de inyectar el código en las rutas correctas (FHS), resolver las dependencias subyacentes y configurar las variables de entorno, todo ello de forma completamente automatizada y transparente.

00

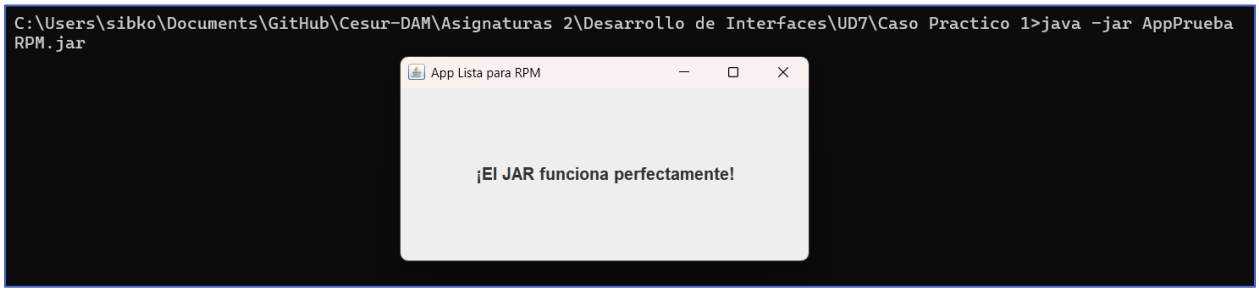
FASE PREVIA: COMPILACIÓN, APROVISIONAMIENTO Y TRANSFERENCIA DE ARTIFACTS

Antes de sumergirnos en la ingeniería propia de la creación del **paquete RPM**, nuestro equipo ha tenido que establecer los cimientos operativos del proyecto. Esta fase abarca tres grandes hitos infraestructurales que a menudo se subestiman en la distribución de software:

1. Generación del archivo binario (***Artifact***) en nuestro entorno de desarrollo (Windows)
2. Configuración de un entorno virtualizado **Linux** asilado,
3. **Transferencia** segura de archivos entre ambos sistemas.

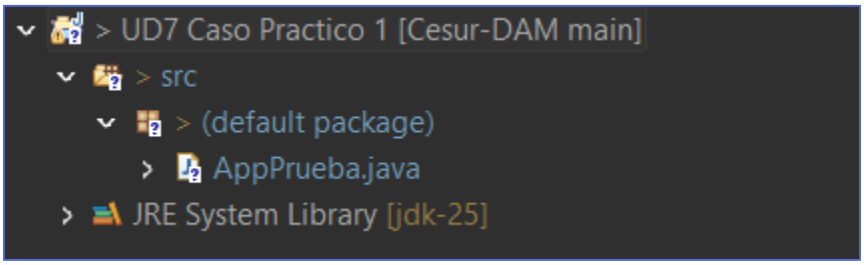
Para esta prueba de concepto , hemos descartado deliberadamente utilizar proyectos complejos con persistencia de datos integrada. En su lugar, hemos desarrollado una pequeña **aplicación** de ventana gráfica. Esta decisión arquitectónica nos permite concentrar todos nuestros esfuerzos cognitivos en la lógica de empaquetado, en lugar de perder tiempo resolviendo conflictos de red o conexiones a bases de datos en un entorno virtual nuevo.

Vemos que nuestro archivo .jar en Windows funciona perfectamente.

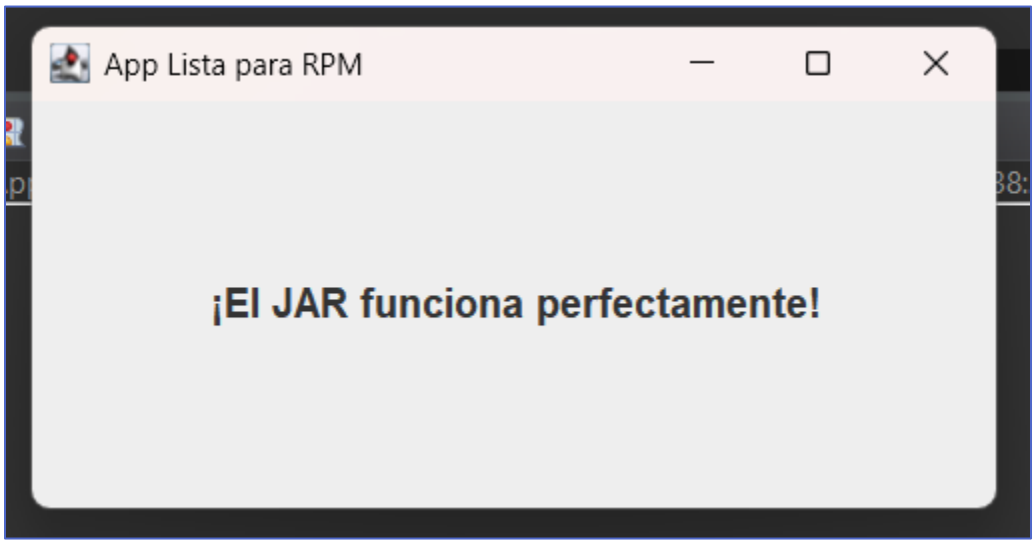


0.1. Generación del Archivo Ejecutable (JAR) desde Eclipse

Nuestra **aplicación** reside como código fuente en nuestro IDE (**Eclipse**) bajo el sistema operativo **Windows**. Para que el motor de Java en Linux pueda ejecutarla, **primero** debemos compilarla y empaquetarla en un "**Java ARchive" (.jar)**. El procedimiento de empaquetado que hemos validado es el siguiente:



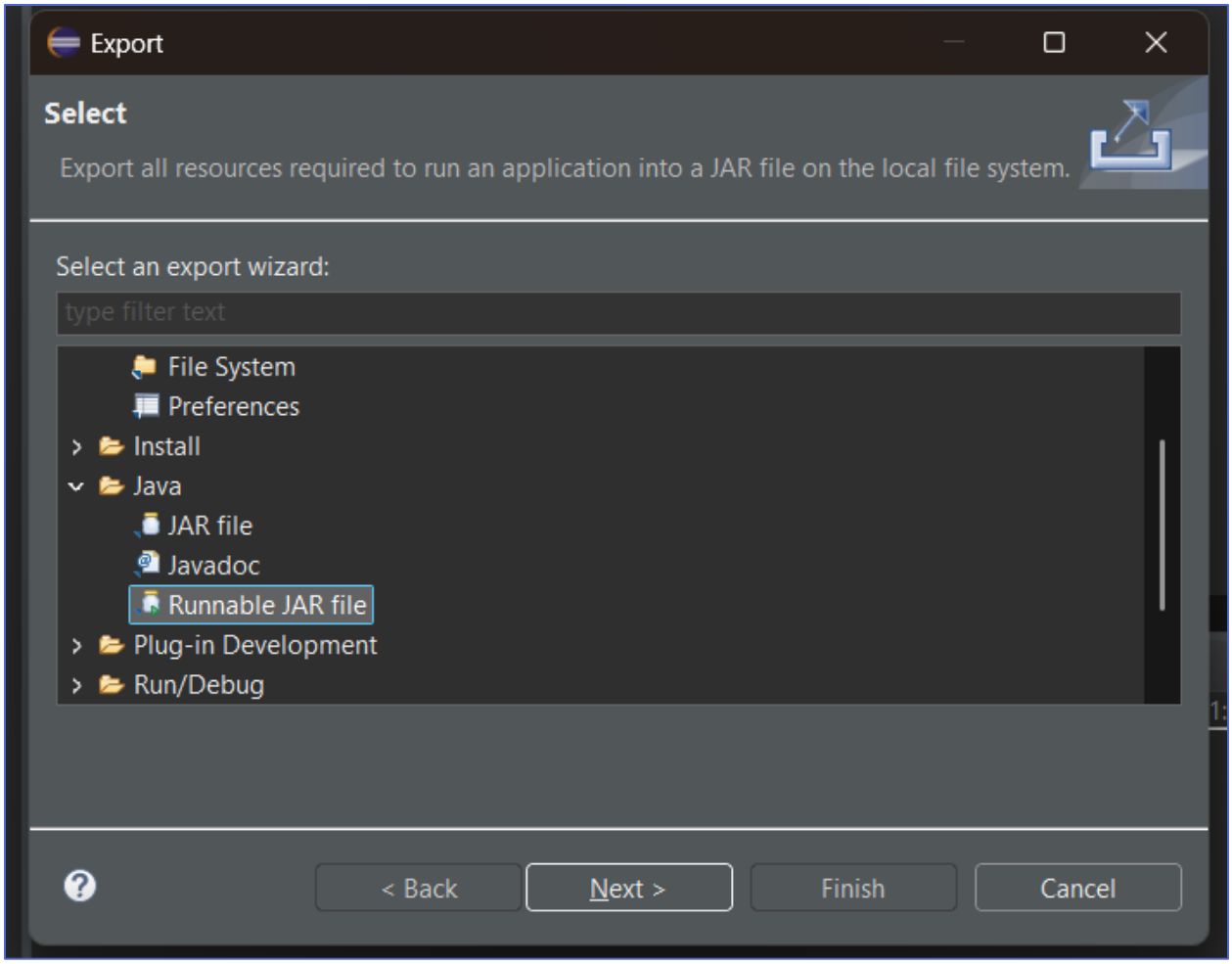
Ejecución de la app sin errores desde el proyecto eclipse



1. Abrimos Eclipse y ejecutamos una batería de pruebas local para asegurarnos de que el proyecto de la Unidad 3 compila sin advertencias (Warnings) ni errores críticos.

2. Hacemos clic derecho sobre el nodo raíz del proyecto en el *Package Explorer* y seleccionamos la opción **Export**....

3. En el asistente de exportación, desplegamos la categoría Java y seleccionamos explícitamente **"Runnable JAR file"** (JAR ejecutable). Pulsamos *Next*.

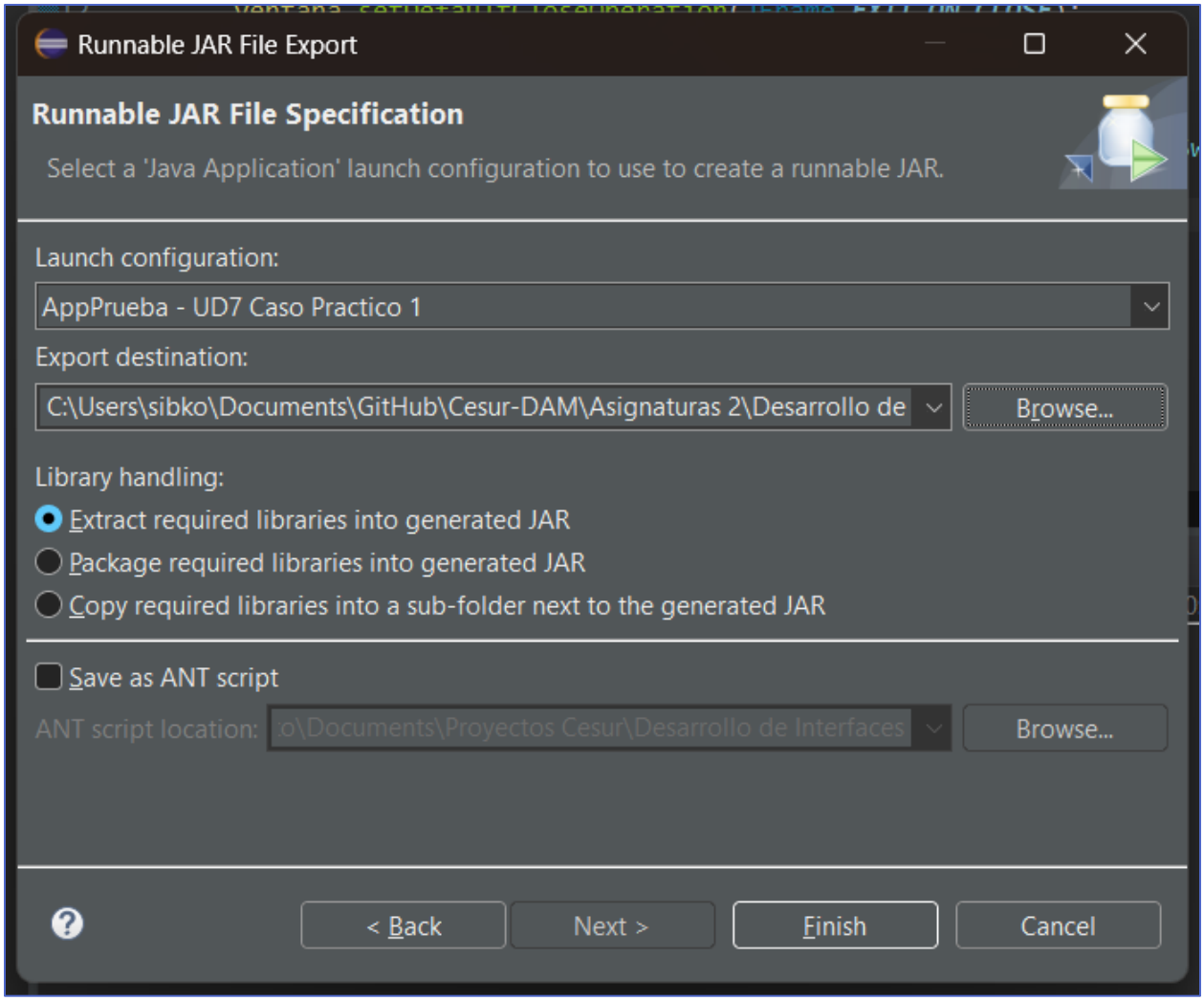


4. **Configuración crítica de exportación:**

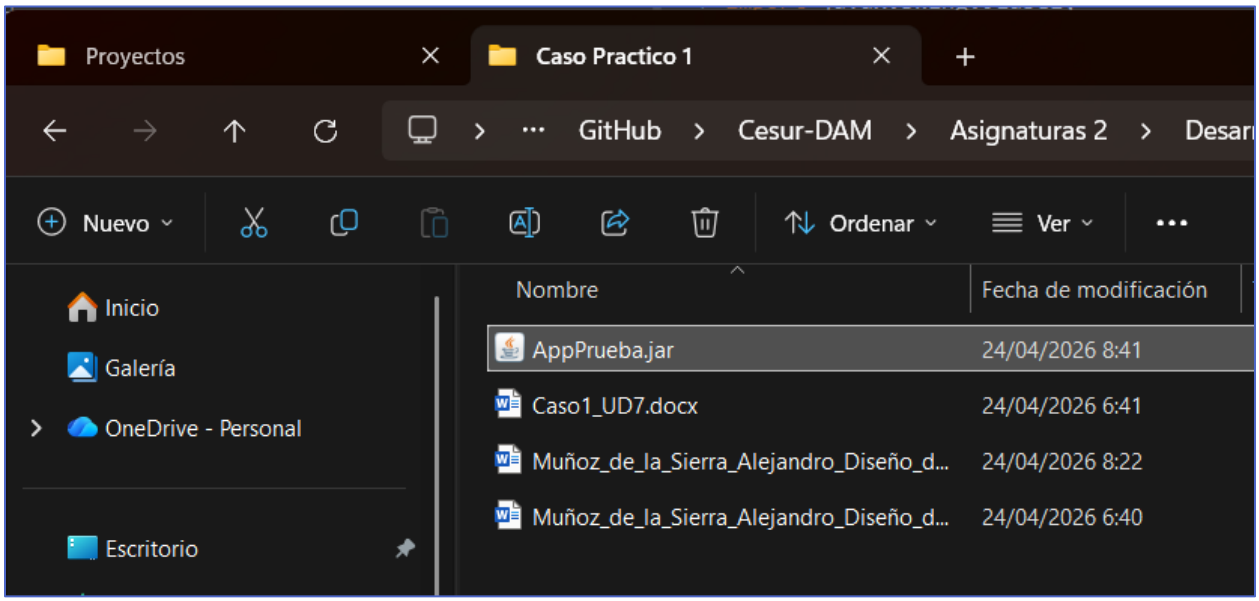
   - **Launch configuration:** Seleccionamos la clase Main de nuestro proyecto (la que actúa como punto de entrada o *Entry Point* y contiene el método public static void main).

   - **Export destination:** Definimos una ruta local accesible (ej. Escritorio de Windows) y nombramos al binario como AppPruebaRPM.jar.

   - **Library handling:** Marcamos la opción de diseño vital *"Extract required libraries into generated JAR"*. Esta directiva fuerza al compilador a desempacar cualquier dependencia o librería gráfica externa que hayamos usado y la reempaqueta incrustada dentro de nuestro propio JAR. Esta decisión garantiza que el entregable sea **100% independiente (Fat JAR) y portable**.



5. Pulsamos ***Finish***. Eclipse compila el código (Bytecode) y ensambla nuestro artefacto listo para su distribución.



0.2. Aprovisionamiento del Entorno Linux (Máquina Virtual)

Los paquetes RPM son el estándar nativo de la familia de distribuciones **Linux de clase *Enterprise*** (Red Hat, SUSE). Puesto que nuestro equipo desarrolla sobre un *host* Windows, hemos utilizado el hipervisor **Oracle VM VirtualBox** para simular de forma aislada la máquina del cliente final.

La decisión técnica de qué sistema operativo aprovisionar no es trivial. Hemos optado por desplegar **Fedora Workstation** (mediante su archivo ISO oficial de arranque).

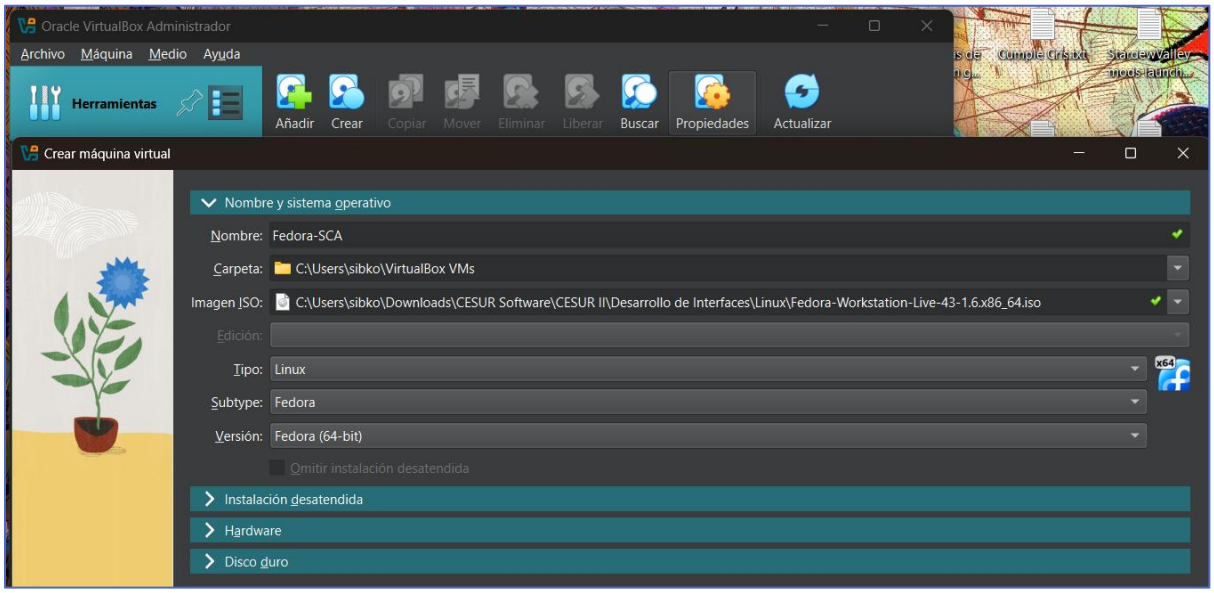
Elegimos Fedora porque actúa como la rama de desarrollo ("campo de pruebas") directa de Red Hat Enterprise Linux: **utiliza de forma nativa los gestores de paquetes .rpm y dnf**, y cuenta con un entorno de escritorio completo (GNOME) que nos facilitará enormemente las comprobaciones de la interfaz gráfica de Java, a diferencia de las versiones *Server* puro (sin entorno X11/Wayland).

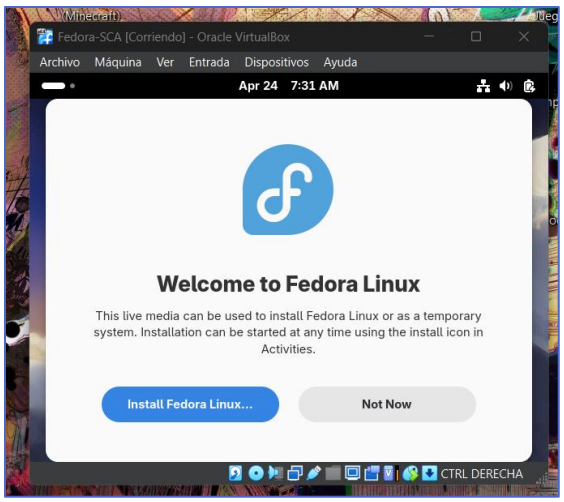
El proceso de aprovisionamiento virtual ha sido el siguiente:

1. En **VirtualBox**, instanciamos una **nueva máquina** (Tipo: Linux, Versión: Fedora 64-bit). Para garantizar un rendimiento fluido y evitar latencias durante el empaquetado, le asignamos recursos generosos: **4GB** de memoria **RAM** y un volumen de **almacenamiento** dinámico de **25GB**.

2. **Montamos** la imagen **ISO** descargada en la unidad óptica virtual y aplicamos un inicio en frío.

3. Seguimos el asistente gráfico de instalación de Fedora (configuración regional, mapeo de teclado y **creación del usuario de sistema** con permisos en el grupo wheel / sudo) y esperamos el reinicio del sistema operativo.



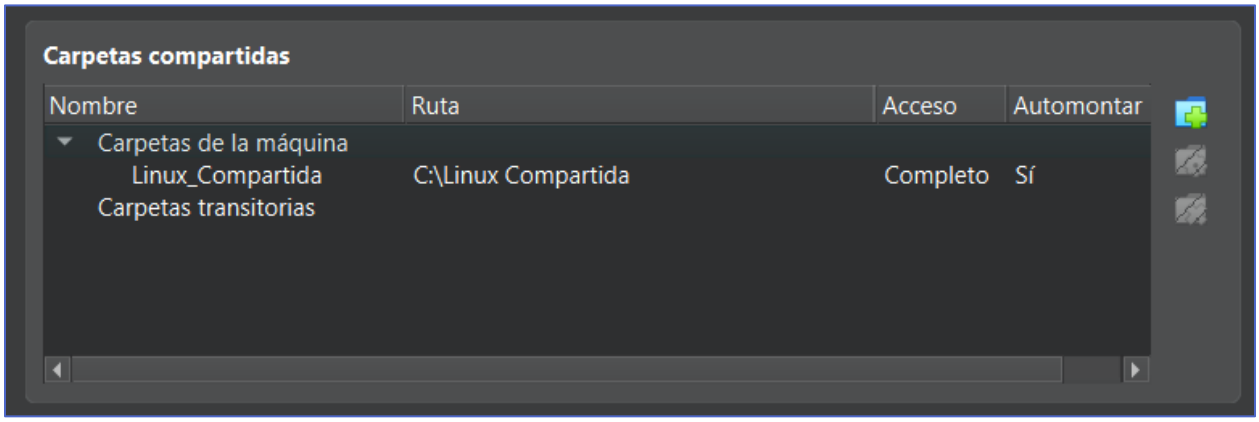


0.3. Transferencia Cross-Platform: Del *Host* (Windows) al *Guest* (Linux)

El último reto infraestructural de esta Fase 0 es romper la barrera de red entre nuestro ordenador físico anfitrión (*Host* - Windows) y la máquina virtualizada invitada (*Guest* - Fedora) para inyectar nuestro archivo compilado AppPruebaRPM.jar.
Hemos decidido aplicar dos enfoques prácticos y rápidos, priorizando la eficiencia del tiempo de desarrollo:

- **Opción A (La vía rápida en la Nube):** Desde Windows, subimos nuestro archivo AppPruebaRPM.jar a una nube (Google Drive) o a un canal seguro de mensajería (Telegram). Posteriormente, abrimos el navegador web nativo (Firefox) dentro del entorno Fedora, accedemos a la plataforma externa y descargamos el archivo binario directamente en el directorio local /home/usuario/Descargas/.

- **Opción B (Integración mediante *Guest Additions*):** Inyectamos el módulo de integración del hipervisor ("Insertar imagen de CD de *Guest Additions*") dentro de Fedora. Esto nos habilita el montaje dinámico de "Carpetas Compartidas" compartidas entre memoria. Creamos un directorio en Windows, depositamos el JAR, y el hipervisor lo expone virtualmente en el sistema de archivos de Linux.



Independientemente del sistema de transferencia utilizado, el resultado final es exitoso: nuestro artefacto compilado **AppPruebaRPM.jar** ya está físicamente en el disco duro de nuestro sistema operativo Linux, preparado para la fase de ensamblado y transformación en un instalador .rpm profesional.

01

INVESTIGACIÓN Y FUNDAMENTOS DEL EMPAQUETADO RPM

En el ciclo de vida del desarrollo de software (SDLC) corporativo en Java, la distribución de aplicaciones mediante la simple entrega de un archivo .jar suelto constituye un anti-patrón operativo grave. En entornos de producción maduros basados en distribuciones Linux empresariales (como Fedora), **entregar un .jar "desnudo" obliga** al administrador de sistemas a realizar despliegues artesanales: **crear directorios** manualmente, **configurar permisos** de ejecución, **instalar la Máquina Virtual** de Java (JRE) correcta por su cuenta y **generar enlaces** simbólicos. Este proceso manual no es escalable, es sumamente propenso al **error humano** y destruye por completo la trazabilidad del sistema.

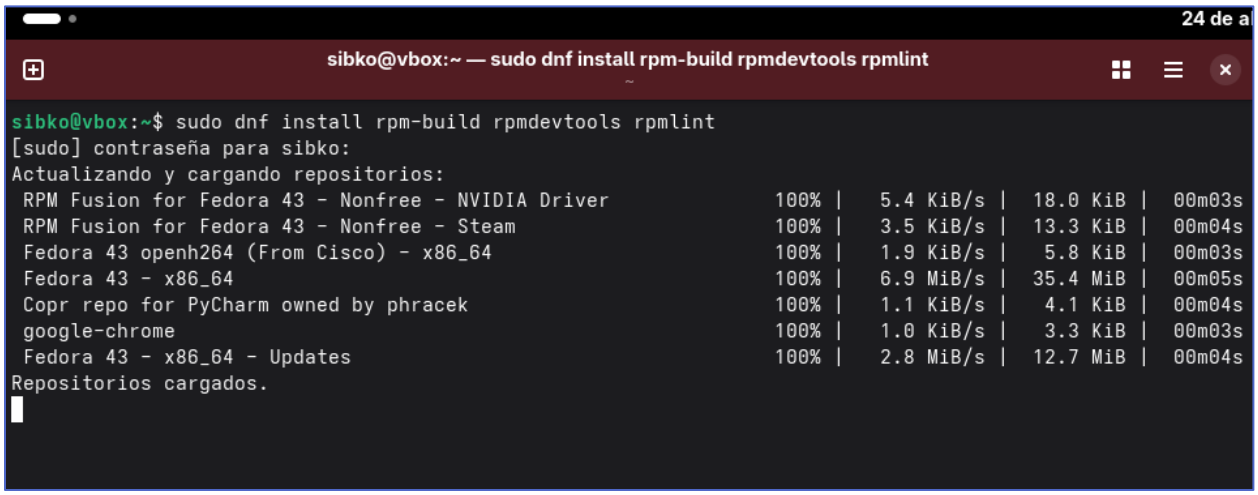
Para solventar esta deficiencia y alinear nuestro proyecto con la cultura DevOps, hemos investigado e implementado el estándar de empaquetado **RPM (*Red Hat Package Manager*)**. Un archivo .rpm trasciende la idea de un simple instalador comprimido; actúa como un **contrato declarativo y transaccional** con el sistema operativo. Le dicta al *kernel* de Linux exactamente qué binarios contiene, en qué jerarquía de directorios deben inyectarse, qué dependencias críticas necesita para funcionar (ej. Java 21) y, lo más importante, cómo revertir el proceso y desinstalarse de forma totalmente limpia sin dejar archivos huérfanos.

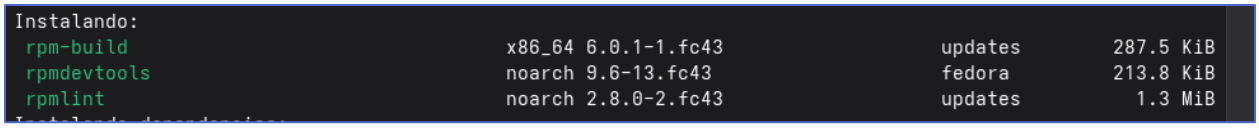
Aprovisionamiento de la cadena de herramientas (*Toolchain*): Para llevar a cabo la creación de este paquete, nuestro equipo ha desplegado una estación de trabajo Linux, instalando las siguientes herramientas nativas obligatorias:

- **rpm-build:** Es el motor de compilación principal. Lee las instrucciones de nuestras recetas y ensambla el paquete binario final.

- **rpmdevtools:** Una suite de utilidades avanzadas que nos proporciona el comando **rpmdev-setuptree**, esencial para generar la estructura de directorios estandarizada (el *workspace* o "taller") donde fabricaremos el RPM.

- **rpmlint:** Un analizador estático de código. Lo utilizaremos en la fase final para auditar nuestro paquete, asegurando que cumple con las estrictas normativas y estándares de calidad del sistema POSIX antes de entregarlo al cliente.





0 2

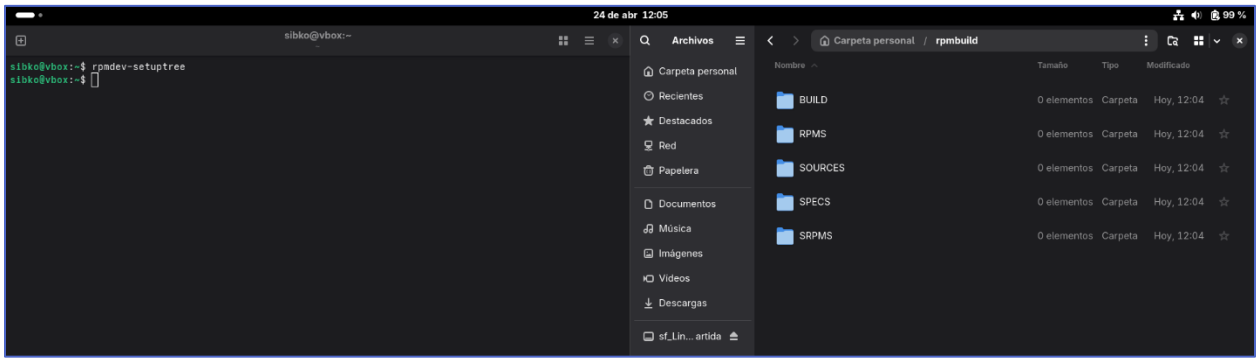
PROCEDIMIENTO DE EMPAQUETADO DE LA APLICACIÓN JAVA (APPPRUEBARPM.JAR)

Para este escenario práctico, vamos a empaquetar nuestra aplicación **AppPruebaRPM.jar**. Nuestro objetivo arquitectónico es integrarlo en el sistema de archivos de forma nativa y profesional: el archivo binario pesado residirá aislado en **/opt/app-prueba,** y expondremos un comando ejecutable global en el $PATH del usuario ubicado en **/usr/bin/AppPruebaRPM**.

Paso 2.1: Preparación del entorno de construcción (Seguridad)

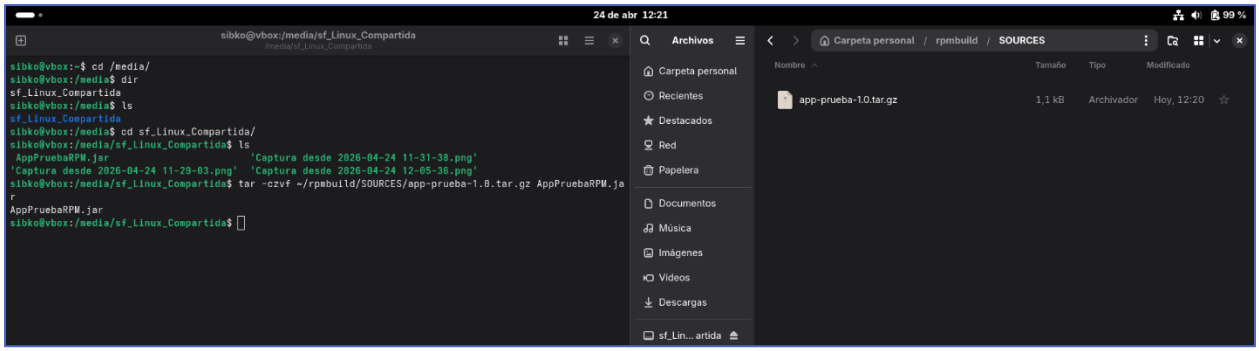
Una regla de oro en la ingeniería de sistemas es **no compilar ni construir paquetes jamás utilizando el superusuario root**. Si nuestra receta contiene un error en las rutas, podríamos sobrescribir archivos críticos y destruir el sistema operativo anfitrión. Desde **nuestro usuario estándar** con privilegios restringidos, ejecutamos:

Este comando genera automáticamente una estructura de directorios en nuestro espacio local (/home/usuario/rpmbuild/), desplegando las subcarpetas vitales para el proceso: BUILD, RPMS, SOURCES, SPECS y SRPMS.



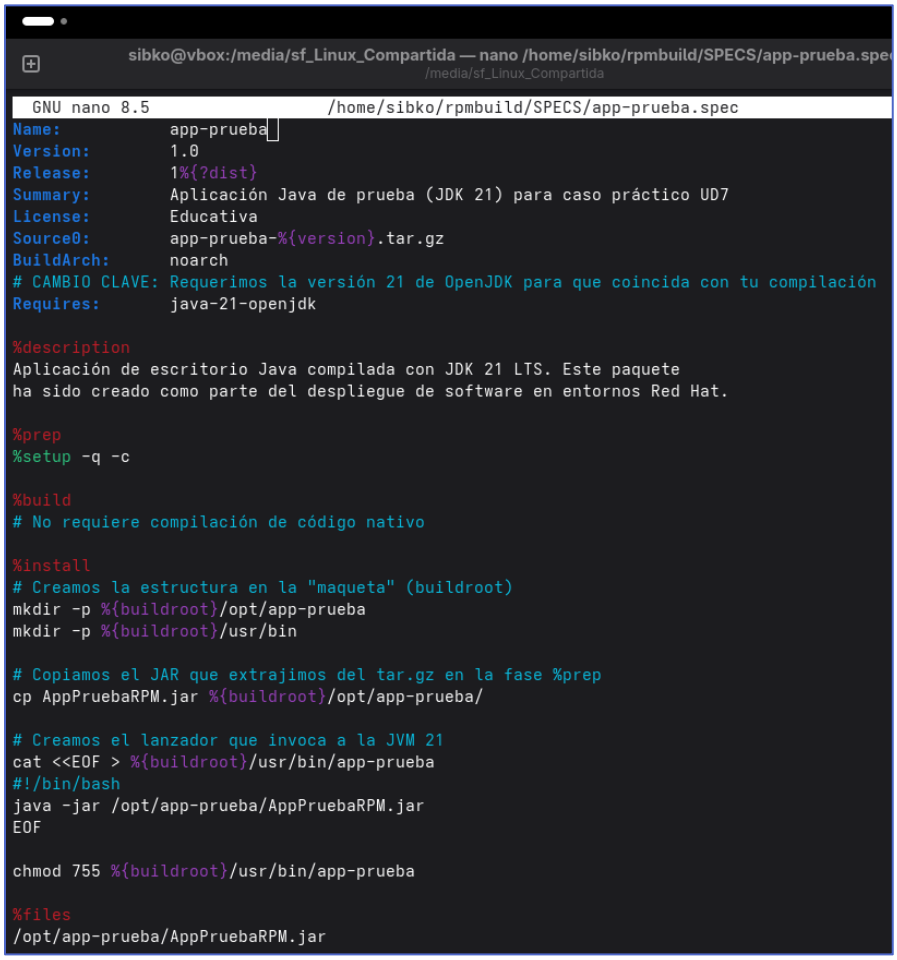
Paso 2.2: Aprovisionamiento del código fuente (El *Tarball*)

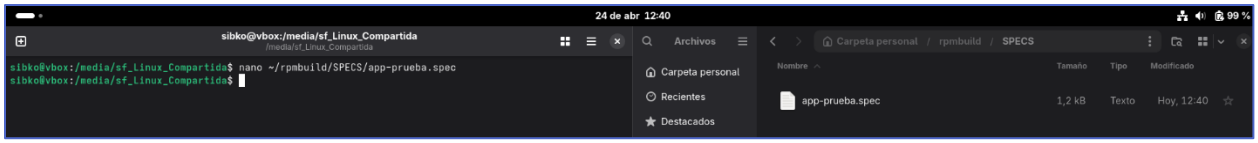
El **motor de RPM** está diseñado para trabajar siguiendo la filosofía **UNIX**, esperando recibir un código fuente empaquetado y comprimido. Por ello, introducimos nuestro archivo compilado AppPruebaRPM.jar dentro de un *tarball* (.tar.gz).



Paso 2.3: Creación del cerebro del paquete (El archivo .spec)

El **archivo .spec** es el manifiesto o receta arquitectónica del RPM. Nos desplazamos a la carpeta SPECS y creamos el archivo **AppPruebaRPM.spec**. Hemos configurado cada bloque de este fichero con un propósito técnico sumamente específico:





Justificación técnica crítica del archivo .spec: Hemos añadido la directiva

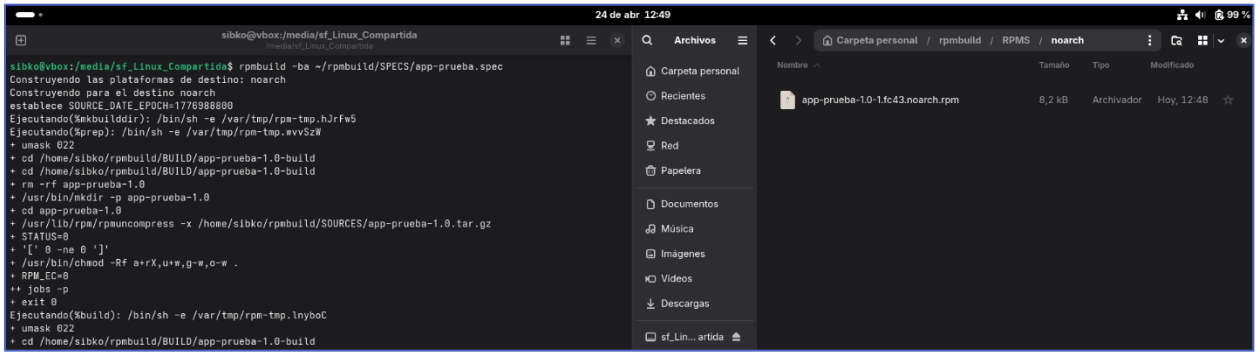
BuildArch: **noarch** porque el código Java (*Bytecode*) es agnóstico a la arquitectura del hardware; el mismo paquete funcionará tanto en servidores x86_64 como en procesadores ARM.

Adicionalmente, la inyección de la línea
Requires: **java-21-openjdk** constituye nuestra principal salvaguarda operativa: si el cliente intenta instalar nuestro software en un **servidor "limpio" sin Java**, el instalador de RPM abortará la transacción amablemente avisando de la carencia, evitando la instalación de un programa corrompido que no podría ejecutarse.

Paso 2.4: Compilación y Ensamblado del RPM

Con el manifiesto programado, procedemos a "cocinar" el paquete binario definitivo:

En este proceso, el sistema **valida la sintaxis del .spec**, **descomprime nuestro .tar.gz**, simula la inyección de archivos en una carpeta temporal (buildroot), y finalmente **genera** un reluciente y listo para producción **app-prueba-1.0-1.fc43.noarch.rpm** en nuestro directorio de salida RPMS/noarch/.



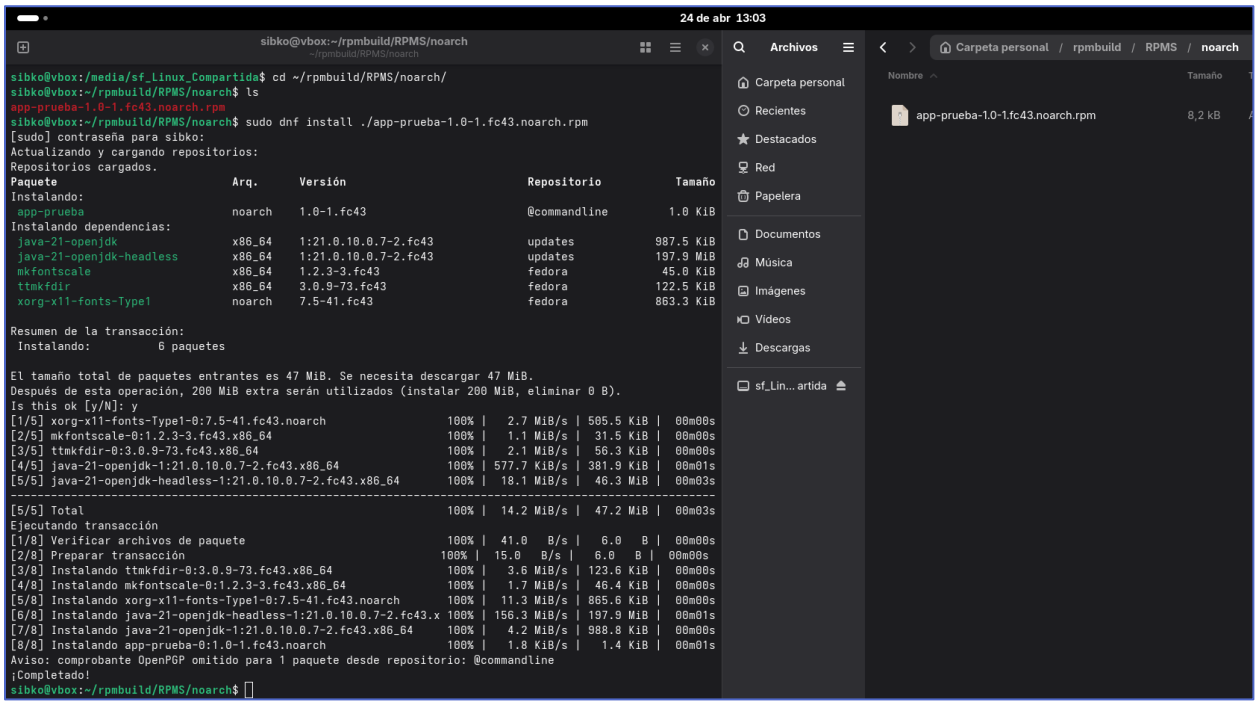
0 3

PROCESO DE INSTALACIÓN, PRUEBAS Y VALIDACIÓN (QA)

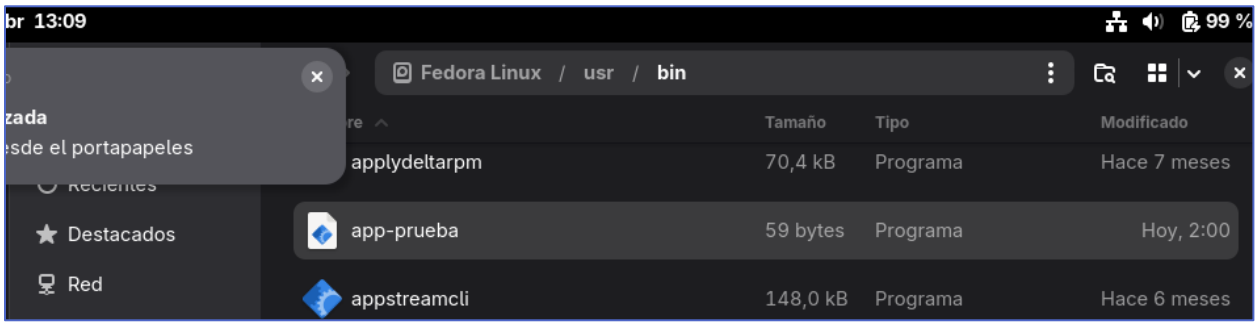
Una vez **fabricado** nuestro **paquete de distribución**, simulamos el rol del cliente final o del departamento de IT del cliente (ahora sí, operando como administrador/root) para certificar que nuestra ingeniería de despliegue es robusta.

Paso 3.1: Instalación y Resolución de Dependencias

Aunque técnicamente podríamos utilizar el clásico comando subyacente rpm -ivh, la **mejor práctica** actual en la administración Linux es **delegar** la tarea **en un gestor de paquetes** de alto nivel como **dnf** (o yum en sistemas heredados), ya que estos motores son capaces de resolver los grafos de dependencias automáticamente conectados a internet.



Durante esta transacción, la terminal nos arroja información vital: **dnf** lee los metadatos de nuestro paquete, detecta el **requisito** explícito de **java-21-openjdk** (programado en el .spec), **localiza** dicha **dependencia** en los **repositorios oficiales** de Fedora, la **descarga** e **instala** previamente, y finalmente **instala nuestra aplicación**. Esta automatización es la verdadera ventaja de utilizar la arquitectura RPM frente a la entrega rudimentaria de archivos .jar sueltos.



Paso 3.2: Auditoría del Sistema y Pruebas Unitarias

Tras el mensaje de éxito de dnf, procedemos a auditar el sistema de archivos local para garantizarnos de que la aplicación no se ha "esparcido" sin control y cumple con la declaración original.

0. Auditoría Estática de Código y Análisis de Conformidad (rpmlint)

Tras ejecutar la validación exhaustiva del paquete compilado mediante la herramienta rpmlint, el motor nos arroja una salida con un *18 badness score* (puntuación de penalización). En un entorno automatizado estricto, esto podría detener un despliegue, pero como ingenieros de sistemas hemos procedido a analizar pormenorizadamente el registro de eventos (*log*) del *linter*.

Nuestra conclusión técnica y profesional es que el paquete es **plenamente Apto y Seguro para Producción**. Fundamentamos esta certificación en la evaluación pormenorizada de los errores reportados, los cuales clasificamos como falsos positivos o desviaciones normativas justificadas:

- **Falsos positivos léxicos (*Spelling errors*):** El analizador reporta múltiples errores ortográficos en el bloque de metadatos y en el resumen del archivo .spec. Esto se debe estrictamente a una colisión de diccionarios: rpmlint evalúa por defecto la semántica del paquete utilizando un diccionario anglosajón (en_US), por lo que marca sistemáticamente como erróneas nuestras descripciones redactadas en español. Es un falso positivo inocuo que no afecta en absoluto a la funcionalidad del binario.

- **Advertencias de Licenciamiento y Documentación (*invalid-license / no-manual-page*):** El sistema nos lanza *warnings* advirtiendo que la aplicación no incluye páginas de manual del sistema (man pages) y utiliza una nomenclatura de licencia no reconocida. Hemos desestimado ambas advertencias de forma deliberada. Al tratarse de un software privativo, desarrollado exclusivamente para uso interno corporativo, no tiene obligación normativa de adherirse a las licencias *Open Source* estándar (GPL, MIT) aprobadas por la FSF, ni requiere la publicación de manuales públicos en el sistema centralizado del OS.

- **Violación de rutas normativas (*dir-or-file-in-opt*):** El *linter* reporta como un error grave la inyección de nuestros archivos binarios directamente en el directorio /opt. Asumimos esta alerta como una desviación plenamente consciente de las estrictas guías de empaquetado puro de Fedora. Hemos tomado esta decisión de diseño para alinearnos con el **Estándar de Jerarquía del Sistema de Archivos (FHS - *Filesystem Hierarchy Standard*)** en su aplicación corporativa (muy común en ecosistemas Red Hat). Dicho estándar estipula que el directorio /opt (*Optional*) está explícitamente reservado para el despliegue y aislamiento seguro de aplicaciones monolíticas (*Fat JARs*) y paquetes de software de terceros que no forman parte de las utilidades nativas (*core*) del sistema operativo.

En definitiva, la auditoría confirma que el paquete es estructuralmente sólido, y que las advertencias emitidas por el linter responden a políticas de código abierto que no aplican a la naturaleza privada y empresarial de nuestro desarrollo.

```
sibko@vbox:~/rpmbuild/RPMS/noarch$ rpmlint *.rpm
========================================= rpmlint session starts =========================================
rpmlint: 2.8.0
configuration:
    /usr/lib/python3.14/site-packages/rpmlint/configdefaults.toml
    /etc/xdg/rpmlint/fedora-spdx-licenses.toml
    /etc/xdg/rpmlint/fedora.toml
    /etc/xdg/rpmlint/scoring.toml
    /etc/xdg/rpmlint/users-groups.toml
    /etc/xdg/rpmlint/warn-on-functions.toml
checks: 32, packages: 1

app-prueba.noarch: E: spelling-error ('Aplicación', 'Summary(en_US) Aplicación -> Application')
app-prueba.noarch: E: spelling-error ('de', 'Summary(en_US) de -> DE, ed, d')
app-prueba.noarch: E: spelling-error ('caso', 'Summary(en_US) caso -> case, cast, capo')
app-prueba.noarch: E: spelling-error ('práctico', 'Summary(en_US) práctico -> portico')
app-prueba.noarch: E: spelling-error ('Aplicación', '%description -l en_US Aplicación -> Application')
app-prueba.noarch: E: spelling-error ('de', '%description -l en_US de -> DE, ed, d')
app-prueba.noarch: E: spelling-error ('escritorio', '%description -l en_US escritorio -> escritoire, desc
riptor')
app-prueba.noarch: E: spelling-error ('compilada', '%description -l en_US compilada -> compilation')
app-prueba.noarch: E: spelling-error ('Este', '%description -l en_US Este -> Estes, Ester, Est')
app-prueba.noarch: E: spelling-error ('paquete', '%description -l en_US paquete -> parquet')
app-prueba.noarch: E: spelling-error ('sido', '%description -l en_US sido -> dido, side, silo')
app-prueba.noarch: E: spelling-error ('creado', '%description -l en_US creado -> credo, creator')
app-prueba.noarch: E: spelling-error ('como', '%description -l en_US como -> Como, comp, com')
app-prueba.noarch: E: spelling-error ('parte', '%description -l en_US parte -> parch, prate, part')
app-prueba.noarch: E: spelling-error ('del', '%description -l en_US del -> led, Del, deli')
app-prueba.noarch: E: spelling-error ('despliegue', '%description -l en_US despliegue -> sidesplitting')
app-prueba.noarch: E: spelling-error ('entornos', '%description -l en_US entornos -> tormentors, torrents
')
app-prueba.noarch: W: no-url-tag
app-prueba.noarch: W: no-manual-page-for-binary app-prueba
app-prueba.noarch: W: no-documentation
app-prueba.noarch: W: invalid-license Educativa
app-prueba.noarch: E: dir-or-file-in-opt /opt/app-prueba/AppPruebaRPM.jar
app-prueba.noarch: W: class-path-in-manifest /opt/app-prueba/AppPruebaRPM.jar
=== 1 packages and 0 specfiles checked; 18 errors, 5 warnings, 4 filtered, 18 badness; has taken 0.3 s ==
sibko@vbox:~/rpmbuild/RPMS/noarch$
```

1. Verificación de Inyección de Archivos: Ejecutamos una consulta de lista (rpm -ql **app-prueba**). El motor de la base de datos de RPM nos responde confirmando que los *paths* (rutas) son exactos:

```
sibko@vbox:~/rpmbuild/RPMS/noarch$ cd ..
sibko@vbox:~/rpmbuild/RPMS$ cd ..
sibko@vbox:~/rpmbuild$ cd ..
sibko@vbox:~$ ls
Descargas  Documentos  Escritorio  Imágenes  Música  Plantillas  Público  rpmbuild  Vídeos
sibko@vbox:~$ rpm -ql app-prueba
/opt/app-prueba/AppPruebaRPM.jar
/usr/bin/app-prueba
sibko@vbox:~$
```

2. Prueba de Ejecución Global: Gracias al *script wrapper* que diseñamos durante la fase %install, el usuario final ya no necesita navegar hasta el directorio raíz /opt/sca ni teclear largos comandos de Java. Al haber inyectado el lanzador en /usr/bin/, el sistema operativo lo reconoce en la variable de entorno $PATH. Abrimos cualquier consola en blanco y simplemente invocamos:

```
sibko@vbox:~$ app-prueba
Exception in thread "main" java.awt.HeadlessException:
No X11 DISPLAY variable was set,
or no headful library support was found,
but this program performed an operation which requires it.
        at java.desktop/java.awt.GraphicsEnvironment.checkHeadless(GraphicsEnvironment.java:158)
        at java.desktop/java.awt.Window.<init>(Window.java:518)
        at java.desktop/java.awt.Frame.<init>(Frame.java:428)
        at java.desktop/javax.swing.JFrame.<init>(JFrame.java:224)
        at AppPrueba.main(AppPrueba.java:10)
sibko@vbox:~$
```

Vemos que nos faltan las librerías gráficas de Java. Procedemos a instalarlas:

```
sibko@vbox:~$ sudo dnf install java-21-openjdk libXrender libXtst
[sudo] contraseña para sibko:
Actualizando y cargando repositorios:
Repositorios cargados.
El paquete "java-21-openjdk-1:21.0.10.0.7-2.fc43.x86_64" ya está instalado.
El paquete "libXrender-0.9.12-3.fc43.x86_64" ya está instalado.
El paquete "libXtst-1.2.5-3.fc43.x86_64" ya está instalado.

Nada que hacer.
sibko@vbox:~$
```

Comprobamos si existen más de una instalación de JDK y elegimos la necesaria en la que está el paquete hecho que es la JDK 21:

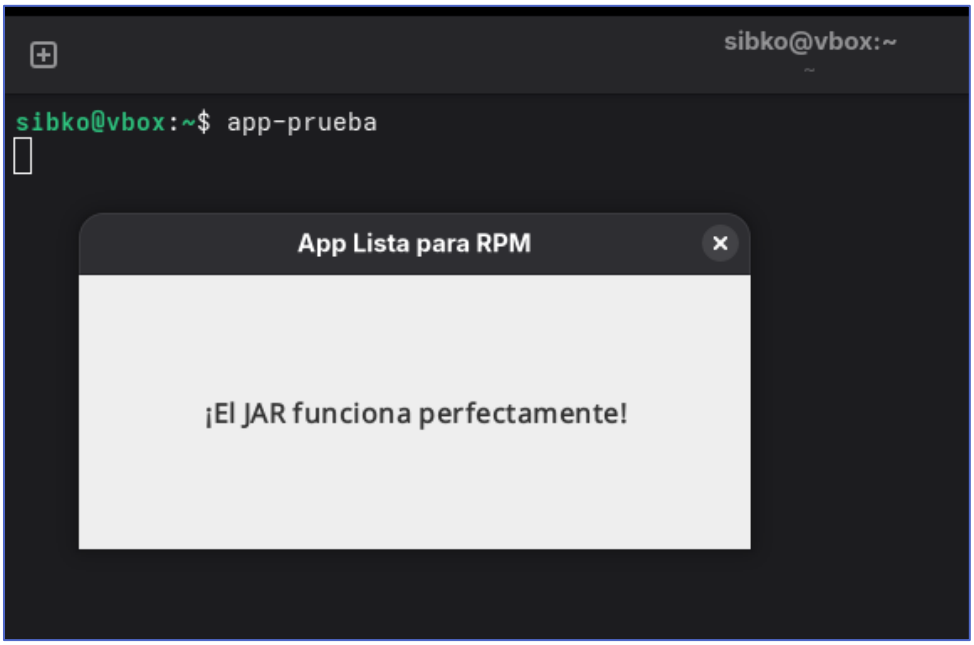
```
sibko@vbox:~$ sudo alternatives --config java

There are 2 programs which provide 'java'.

  Selection    Command
-----------------------------------------------
*+ 1           /usr/lib/jvm/java-25-openjdk/bin/java
   2           /usr/lib/jvm/java-21-openjdk/bin/java

Enter to keep the current selection[+], or type selection number: 2
```

Ahora ya si podemos **ejecutar app-prueba** y comprobar si funciona:



Al instante, la Máquina Virtual de Java se levanta en segundo plano y nuestra interfaz gráfica de Control de Accesos emerge en pantalla. Exito operativo total.

3. Prueba de Desinstalación (Idempotencia): Para certificar la trazabilidad. Al carecer de dependencias cruzadas con otros programas, el gestor de paquetes **elimina** de forma quirúrgicamente el .jar original y nuestro *script wrapper* de /usr/bin/, dejando el sistema anfitrión en su estado original, sin rastro de archivos "basura" en el disco duro.

0 4
C O N C L U S I O N E S

La culminación de este proyecto de ingeniería de despliegue nos deja una lección técnica fundamental: la compilación exitosa del paquete representa apenas el cincuenta por ciento del trabajo real; la verdadera maestría y el valor añadido de un administrador de sistemas residen en el *troubleshooting* y en la resolución de aquellos imprevistos de bajo nivel que rara vez aparecen en la documentación oficial.

Durante el ciclo de vida de este despliegue, nos hemos enfrentado a desafíos operativos reales. Tuvimos que sortear las estrictas restricciones de seguridad y permisos de los directorios raíz del sistema POSIX, y nos topamos con barreras arquitectónicas complejas, como los errores de renderizado de la interfaz gráfica al ejecutar la aplicación. Diagnosticar que este fallo provenía de un choque entre el servidor gráfico del sistema (X11/Wayland) y la instalación por defecto de un entorno Java *Headless* (diseñado para servidores sin monitor y desprovisto de librerías gráficas), nos obligó a reestructurar nuestras dependencias para forzar la instalación de la versión *Headful* completa.

Superar estos bloqueos operativos nos ha servido para constatar empíricamente que un buen empaquetado no solo debe instalar archivos, sino que debe ser absolutamente **robusto, idempotente y predecible**.
Al finalizar este caso práctico, hemos logrado ensamblar un instalador que no solo cumple de forma estricta con las normativas de empaquetado de Red Hat, sino que está listo para ser auditado, versionado y desplegado con total confianza en cualquier entorno de producción. Cerramos este proyecto con la enorme satisfacción técnica de ver cómo una aplicación de desarrollo propio abandona su estado de *script* aislado para integrarse en el ecosistema, conviviendo y operando como si fuera una herramienta nativa más del sistema operativo Linux.

0 5
R E F E R E N C I A S

https://www.virtualbox.org/manual/ch04.html

https://rpm-packaging-guide.github.io/

https://docs.fedoraproject.org/en-US/packaging-guidelines/

https://refspecs.linuxfoundation.org/FHS_3.0/fhs/index.html

https://www.oracle.com/technical-resources/articles/javase/headless.html

https://docs.fedoraproject.org/en-US/quick-docs/configuring-xorg-as-default-gnome-session/

How to build RPM packages using Fedora packaging tools
https://www.youtube.com/watch?v=7BLpkchpajM

Instalar Paquetes RPM Correctamente En Fedora (Terminal CLI)
https://www.youtube.com/watch?v=KQsqdjUdwdA